Summarize

Chapter 1. What Happens When There Are No "Best Practices"?

Why does a technologist like a software architect present at a conference or write a book? Because they have discovered what is colloquially known as a "best practice," a term so overused that those who speak it increasingly experience backlash. Regardless of the term, technologists write books when they have figured out a novel solution to a general problem and want to broadcast it to a wider audience.

But what happens for that vast set of problems that have no good solutions? Entire classes of problems exist in software architecture that have no general good solutions, but rather present one messy set of trade-offs cast against an (almost) equally messy set.

Software developers build outstanding skills in searching online for solutions to a current problem. For example, if they need to figure out how to configure a particular tool in their environment, expert use of Google finds the answer.

But that's not true for architects.

For architects, many problems present unique challenges because they conflate the exact environment and circumstances of your organization—what are the chances that someone has encountered exactly this scenario *and* blogged it or posted it on Stack Overflow?

Architects may have wondered why so few books exist about architecture compared to technical topics like frameworks, APIs, and so on. Architects rarely experience common problems but constantly struggle with decision making in novel situations. For architects, every problem is a snowflake. In many cases, the problem is novel not just within a particular organization but rather throughout the world. No books or conference sessions exist for those problems!

Architects shouldn't constantly seek out silver-bullet solutions to their problems; they are as rare now as in 1986, when Fred Brooks coined the term:

> *There is no single development, in either technology or management technique, which by itself promises even one order of magnitude [tenfold] improvement within a decade in productivity, in reliability, in simplicity.*
>
> —Fred Brooks from "No Silver Bullet"

Because virtually every problem presents novel challenges, the real job of an architect lies in their ability to objectively determine and assess the set of trade-offs on either side of a consequential decision to resolve it as well as possible. The authors don't talk about "best solutions" (in this book or in the real world) because "best" implies that an architect has managed to maximize all the possible competing factors within the design. Instead, our tongue-in-cheek advice is as follows:

TIP

Don't try to find the *best* design in software architecture; instead, strive for the *least worst* combination of trade-offs.

Often, the best design an architect can create is the least worst collection of trade-offs—no single architecture characteristics excels as it would alone, but the balance of all the competing architecture characteristics promote project success.

Which begs the question: "How can an architect *find* the least worst combination of trade-offs (and document them effectively)?" This book is primarily about decision making, enabling architects to make better decisions when confronted with novel situations.

Why "The Hard Parts"?

Why did we call this book *Software Architecture: The Hard Parts?* Actually, the "hard" in the title performs double duty. First, *hard* connotes *difficult*, and architects constantly face difficult problems that literally (and figuratively) no one has faced before, involving numerous technology decisions with long-term implications layered on top of the interpersonal and political environment where the decision must take place.

Second, *hard* connotes *solidity*—just as in the separation of *hardware* and *software*, the *hard* one should change much less because it provides the foundation for the *soft* stuff.

Similarly, architects discuss the distinction between *architecture* and *design*, where the former is structural and the latter is more easily changed. Thus, in this book, we talk about the foundational parts of architecture.

The definition of software architecture itself has provided many hours of non-productive conversation among its practitioners. One favorite tongue-in-cheek definition is that "software architecture is the *stuff* that's hard to change later." That *stuff* is what our book is about.

Giving Timeless Advice About Software Architecture

The software development ecosystem constantly and chaotically shifts and grows. Topics that were all the rage a few years ago have either been subsumed by the ecosystem and disappeared or replaced by something different/better. For example, 10 years ago, the predominant architecture style for large enterprises was orchestration-driven, service-oriented architecture. Now, virtually no one builds in that architecture style anymore (for reasons we'll uncover along the way); the current favored style for many distributed systems is microservices. How and why did that transition happen?

When architects look at a particular style (especially a historical one), they must consider the constraints in place that lead to that architecture becoming dominant. At the time, many companies were merging to become *enterprises*, with all the attendant integration woes that come with that transition. Additionally, open source wasn't a viable option (often for political rather than technical reasons) for large companies. Thus, architects emphasized shared resources and centralized orchestration as a solution.

However, in the intervening years, open source and Linux became viable alternatives, making operating systems *commercially* free. However, the real tipping point occurred when Linux became *operationally* free with the advent of tools like Puppet and Chef, which allowed development teams to programmatically spin up their environments as part of an automated build. Once that capability arrived, it fostered an architectural revolution with microservices and the quickly emerging infrastructure of containers and orchestration tools like Kubernetes.

This illustrates that the software development ecosystem expands and evolves in completely unexpected ways. One new capability leads to another one, which unexpectedly creates new capabilities. Over the course of time, the ecosystem completely replaces itself, one piece at a time.

This presents an age-old problem for authors of books about technology generally and software architecture specifically—how can we write something that isn't old immediately?

We don't focus on technology or other implementation details in this book. Rather, we focus on *how* architects make decisions, and how to objectively weigh trade-offs when presented with novel situations. We use contemporaneous scenarios and examples to provide details and context, but the underlying principles focus on trade-off analysis and decision making when faced with new problems.

The Importance of Data in Architecture

> *Data is a precious thing and will last longer than the systems themselves.*
>
> —Tim Berners-Lee

For many in architecture, data is everything. Every enterprise building any system must deal with data, as it tends to live much longer than systems or architecture, requiring diligent thought and design. However, many of the instincts of data architects to build tightly coupled systems create conflicts within modern distributed architectures. For example, architects and DBAs must ensure that business data survives the breaking apart of monolith systems and that the business can still derive value from its data regardless of architecture undulations.

It has been said that *data is the most important asset in a company*. Businesses want to extract value from the data that they have and are finding new ways to deploy data in decision making. Every part of the enterprise is now data driven, from servicing existing customers, to acquiring new customers, increasing customer retention, improving products, predicting sales, and other trends. This reliance on data means that all software architecture is in the service of data, ensuring the right data is available and usable by all parts of the enterprise.

The authors built many distributed systems a few decades ago when they first became popular, yet decision making in modern microservices seems more difficult, and we wanted to figure out why. We eventually realized that, back in the early days of distributed architecture, we mostly still persisted data in a single relational database. However, in microservices and the philosophical adherence to a *bounded context* from Domain-Driven Design, as a way of limiting the scope of implementation detail coupling, data has moved to an architectural concern, along with transactionality. Many of the hard

parts of modern architecture derive from tensions between data and architecture concerns, which we untangle in both Part I and Part II.

One important distinction that we cover in a variety of chapters is the separation between *operational* versus *analytical* data:

Operational data

> Data used for the operation of the business, including sales, transactional data, inventory, and so on. This data is what the company runs on—if something interrupts this data, the organization cannot function for very long. This type of data is defined as *Online Transactional Processing* (OLTP), which typically involves inserting, updating, and deleting data in a database.

Analytical data

> Data used by data scientists and other business analysts for predictions, trending, and other business intelligence. This data is typically not transactional and often not relational—it may be in a graph database or snapshots in a different format than its original transactional form. This data isn't critical for the day-to-day operation but rather for the long-term strategic direction and decisions.

We cover the impact of both operational and analytical data throughout the book.

Architectural Decision Records

One of the most effective ways of documenting architecture decisions is through *Architectural Decision Records* (ADRs). ADRs were first evangelized by Michael Nygard in a blog post and later marked as "adopt" in the Thoughtworks Technology Radar. An ADR consists of a short text file (usually one to two pages long) describing a specific architecture decision. While ADRs can be written using plain text, they are usually written in some sort of text document format like AsciiDoc or Markdown. Alternatively, an ADR can also be written using a wiki page template. We devoted an entire chapter to ADRs in our previous book, *Fundamentals of Software Architecture* (O'Reilly).

We will be leveraging ADRs as a way of documenting various architecture decisions made throughout the book. For each architecture decision, we will be using the following ADR format with the assumption that each ADR is approved:

ADR: A short noun phrase containing the architecture decision

Context
In this section of the ADR we will add a short one- or two-sentence description of the problem, and list the alternative solutions.

Decision
In this section we will state the architecture decision and provide a detailed justification of the decision.

Consequences
In this section of the ADR we will describe any consequences after the decision is applied, and also discuss the trade-offs that were considered.

A list of all the Architectural Decision Records created in this book can be found in Appendix B.

Documenting a decision is important for an architect, but governing the proper use of the decision is a separate topic. Fortunately, modern engineering practices allow automating many common governance concerns by using architecture fitness functions.

Architecture Fitness Functions

Once an architect has identified the relationship between components and codified that into a design, how can they make sure that the implementers will adhere to that design? More broadly, how can architects ensure that the design principles they define become reality if they aren't the ones to implement them?

These questions fall under the heading of *architecture governance*, which applies to any organized oversight of one or more aspects of software development. As this book primarily covers architecture structure, we cover how to automate design and quality principles via fitness functions in many places.

Software development has slowly evolved over time to adapt unique engineering practices. In the early days of software development, a manufacturing metaphor was commonly applied to software practices, both in the large (like the Waterfall development process) and small (integration practices on projects). In the early 1990s, a rethinking of software development engineering practices, lead by Kent Beck and the other engineers on the C3 project, called eXtreme Programming (XP), illustrated the importance of incremental feedback and automation as key enablers of software development productivity.

In the early 2000s, the same lessons were applied to the intersection of software development and operations, spawning the new role of DevOps and automating many formerly manual operational chores. Just as before, automation allows teams to go faster because they don't have to worry about things breaking without good feedback. Thus, *automation* and *feedback* have become central tenets for effective software development.

Consider the environments and situations that lead to breakthroughs in automation. In the era before continuous integration, most software projects included a lengthy integration phase. Each developer was expected to work in some level of isolation from others, then integrate all the code at the end into an integration phase. Vestiges of this practice still linger in version control tools that force branching and prevent continuous integration. Not surprisingly, a strong correlation existed between project size and the pain of the integration phase. By pioneering continuous integration, the XP team illustrated the value of rapid, continuous feedback.

The DevOps revolution followed a similar course. As Linux and other open source software became "good enough" for enterprises, combined with the advent of tools that allowed programmatic definition of (eventually) virtual machines, operations personnel realized they could automate machine definitions and many other repetitive tasks.

In both cases, advances in technology and insights led to automating a recurring job that was handled by an expensive role—which describes the current state of architecture governance in most organizations. For example, if an architect chooses a particular architecture style or communication medium, how can they make sure that a developer implements it correctly? When done manually, architects perform code reviews or perhaps hold architecture review boards to assess the state of governance. However, just as in manually configuring computers in operations, important details can easily fall through superficial reviews.

Using Fitness Functions

In the 2017 book *Building Evolutionary Architectures* (O'Reilly), the authors (Neal Ford, Rebecca Parsons, and Patrick Kua) defined the concept of an *architectural fitness function*: any mechanism that performs an objective integrity assessment of some architecture characteristic or combination of architecture characteristics. Here is a point-by-point breakdown of that definition:

Any mechanism

Architects can use a wide variety of tools to implement fitness functions; we will show numerous examples throughout the book. For example, dedicated testing libraries exist to test architecture structure, architects can use monitors to test operational architecture characteristics such as performance or scalability, and chaos engineering frameworks test reliability and resiliency.

Objective integrity assessment

One key enabler for automated governance lies with objective definitions for architecture characteristics. For example, an architect can't specify that they want a "high performance" website; they must provide an object value that can be measured by a test, monitor, or other fitness function.

Architects must watch out for *composite architecture characteristics*—ones that aren't objectively measurable but are really composites of other measurable things. For example, "agility" isn't measurable, but if an architect starts pulling the broad term *agility* apart, the goal is for teams to be able to respond quickly and confidently to change, either in ecosystem or domain. Thus, an architect can find measurable characteristics that contribute to agility: deployability, testability, cycle time, and so on. Often, the lack of ability to measure an architecture characteristic indicates too vague a definition. If architects strive toward measurable properties, it allows them to automate fitness function application.

Some architecture characteristic or combination of architecture characteristics

This characteristic describes the two scopes for fitness functions:

Atomic

These fitness functions handle a single architecture characteristic in isolation. For example, a fitness function that checks for component cycles within a codebase is atomic in scope.

Holistic

Holistic fitness functions validate a combination of architecture characteristics. A complicating feature of architecture characteristics is the synergy they sometimes exhibit with other architecture characteristics. For example, if an architect wants to improve security, a good chance exists that it will affect performance. Similarly, scalability and elasticity are sometimes at odds—supporting a large number of concurrent users can make handling sudden bursts more difficult. Holistic fitness functions exercise a combina-

tion of interlocking architecture characteristics to ensure that the combined effect won't negatively affect the architecture.

An architect implements fitness functions to build protections around unexpected change in architecture characteristics. In the Agile software development world, developers implement unit, functional, and user acceptance tests to validate different dimensions of the *domain* design. However, until now, no similar mechanism existed to validate the *architecture characteristics* part of the design. In fact, the separation between fitness functions and unit tests provides a good scoping guideline for architects. Fitness functions validate architecture characteristics, not domain criteria; unit tests are the opposite. Thus, an architect can decide whether a fitness function or unit test is needed by asking the question: "Is any domain knowledge required to execute this test?" If the answer is "yes," then a unit/function/user acceptance test is appropriate; if "no," then a fitness function is needed.

For example, when architects talk about *elasticity*, it's the ability of the application to withstand a sudden burst of users. Notice that the architect doesn't need to know any details about the domain—this could be an ecommerce site, an online game, or something else. Thus, *elasticity* is an architectural concern and within the scope of a fitness function. If on the other hand the architect wanted to validate the proper parts of a mailing address, that is covered via a traditional test. Of course, this separation isn't purely binary—some fitness functions will touch on the domain and vice versa, but the differing goals provide a good way to mentally separate them.

Here are a couple of examples to make the concept less abstract.

One common architect goal is to maintain good internal structural integrity in the codebase. However, malevolent forces work against the architect's good intentions on many platforms. For example, when coding in any popular Java or .NET development environment, as soon as a developer references a class not already imported, the IDE helpfully presents a dialog asking the developer if they would like to auto-import the reference. This occurs so often that most programmers develop the habit of swatting the auto-import dialog away like a reflex action.

However, arbitrarily importing classes or components among one another spells disaster for modularity. For example, Figure 1-1 illustrates a particularly damaging anti-pattern that architects aspire to avoid.

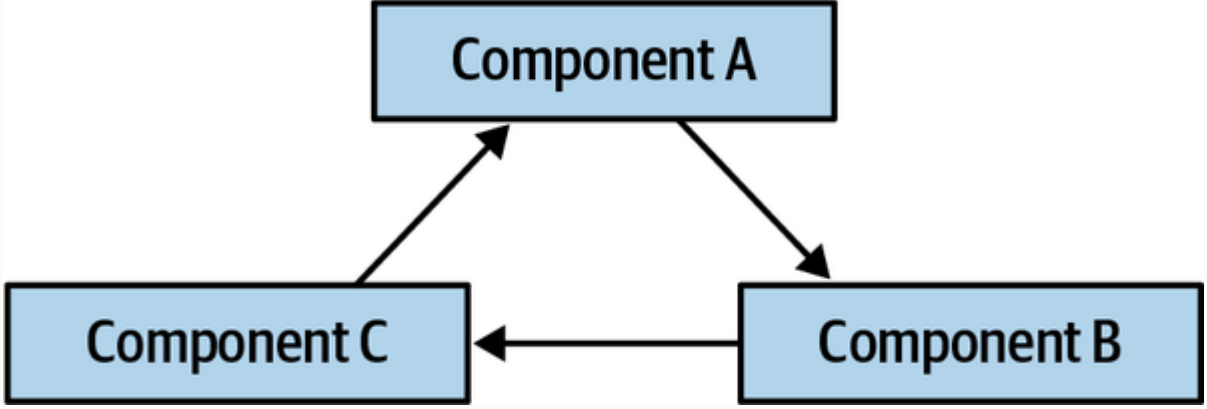


Figure 1-1. Cyclic dependencies between components

In this anti-pattern, each component references something in the others. Having a network of components such as this damages modularity because a developer cannot reuse a single component without also bringing the others along. And, of course, if the other components are coupled to other components, the architecture tends more and more toward the Big Ball of Mud anti-pattern. How can architects govern this behavior without constantly looking over the shoulders of trigger-happy developers? Code reviews help but happen too late in the development cycle to be effective. If an architect allows a development team to rampantly import across the codebase for a week until the code review, serious damage has already occurred in the codebase.

The solution to this problem is to write a fitness function to avoid component cycles, as shown in Example 1-1.

Example 1-1. Fitness function to detect component cycles

```
public class CycleTest {
    private JDepend jdepend;

    @BeforeEach
    void init() {
        jdepend = new JDepend();
        jdepend.addDirectory("/path/to/project/persistence/classes");
        jdepend.addDirectory("/path/to/project/web/classes");
        jdepend.addDirectory("/path/to/project/thirdpartyjars");
    }

    @Test
    void testAllPackages() {
        Collection packages = jdepend.analyze();
        assertEquals("Cycles exist", false, jdepend.containsCycles());
    }
}
```

In the code, an architect uses the metrics tool JDepend to check the dependencies between packages. The tool understands the structure of Java packages and fails the test if any cycles exist. An architect can wire this test into the continuous build on a project and stop worrying about the accidental introduction of cycles by trigger-happy developers. This is a great example of a fitness function guarding the important rather than urgent practices of software development: it's an important concern for architects, yet has little impact on day-to-day coding.

Example 1-1 shows a very low-level, code-centric fitness function. Many popular code hygiene tools (such as SonarQube) implement many common fitness functions in a turnkey manner. However, architects may also want to validate the macro structure of the architecture as well as the micro. When designing a layered architecture such as the one in Figure 1-2, the architect defines the layers to ensure separation of concerns.

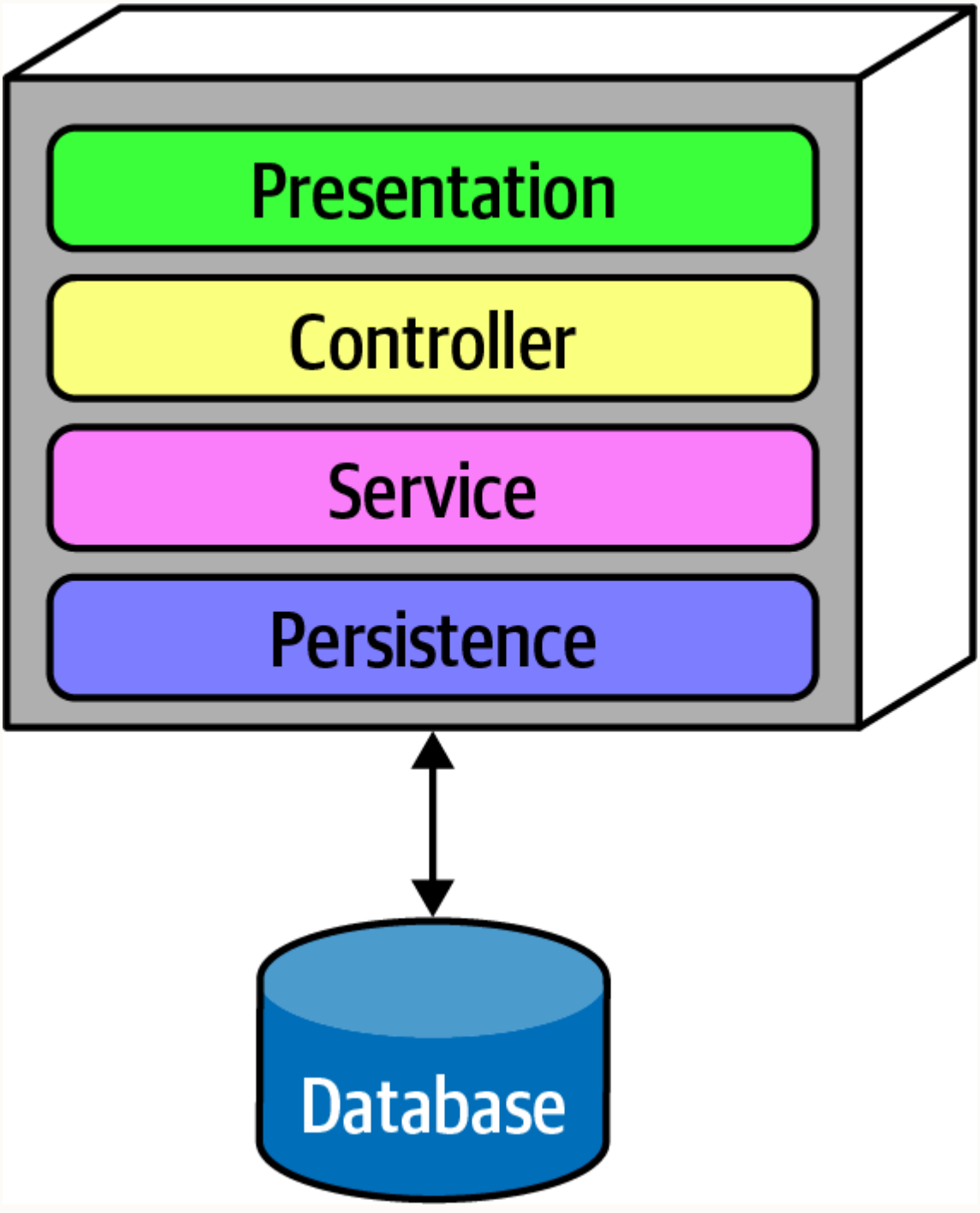


Figure 1-2. Traditional layered architecture

However, how can the architect ensure that developers will respect these layers? Some developers may not understand the importance of the patterns, while others may adopt a "better to ask forgiveness than permission" attitude because of some overriding local concern, such as performance. But allowing implementers to erode the reasons for the architecture hurts the long-term health of the architecture.

ArchUnit allows architects to address this problem via a fitness function, shown in Example 1-2.

Example 1-2. ArchUnit fitness function to govern layers

```
layeredArchitecture()
    .layer("Controller").definedBy("..controller..")
    .layer("Service").definedBy("..service..")
    .layer("Persistence").definedBy("..persistence..")

    .whereLayer("Controller").mayNotBeAccessedByAnyLayer()
    .whereLayer("Service").mayOnlyBeAccessedByLayers("Controller")
    .whereLayer("Persistence").mayOnlyBeAccessedByLayers("Service")
```

In Example 1-2, the architect defines the desirable relationship between layers and writes a verification fitness function to govern it. This allows an architect to establish architecture principles outside the diagrams and other informational artifacts, and verify them on an ongoing basis.

A similar tool in the .NET space, NetArchTest, allows similar tests for that platform. A layer verification in C# appears in Example 1-3.

Example 1-3. NetArchTest for layer dependencies

```
// Classes in the presentation should not directly reference repositories
var result = Types.InCurrentDomain()
    .That()
    .ResideInNamespace("NetArchTest.SampleLibrary.Presentation")
    .ShouldNot()
    .HaveDependencyOn("NetArchTest.SampleLibrary.Data")
    .GetResult()
    .IsSuccessful;
```

Tools continue to appear in this space with increasing degrees of sophistication. We will continue to highlight many of these techniques as we illustrate fitness functions along-

side many of our solutions.

Finding an objective outcome for a fitness function is critical. However, *objective* doesn't imply *static*. Some fitness functions will have noncontextual return values, such as true/false or a numeric value such as a performance threshold. However, other fitness functions (deemed *dynamic*) return a value based on some context. For example, when measuring *scalability*, architects measure the number of concurrent users and also generally measure the performance for each user. Often, architects design systems so that as the number of users goes up, performance per user declines slightly—but doesn't fall off a cliff. Thus, for these systems, architects design performance fitness functions that take into account the number of concurrent users. As long as the measure of an architecture characteristic is objective, architects can test it.

While most fitness functions should be automated and run continually, some will necessarily be manual. A manual fitness function requires a person to handle the validation. For example, for systems with sensitive legal information, a lawyer may need to review changes to critical parts to ensure legality, which cannot be automated. Most deployment pipelines support manual stages, allowing teams to accommodate manual fitness functions. Ideally, these are run as often as reasonably possible—a validation that doesn't run can't validate anything. Teams execute fitness functions either on demand (rarely) or as part of a continuous integration work stream (most common). To fully achieve the benefit of validations such as fitness functions, they should be run continually.

Continuity is important, as illustrated in this example of enterprise-level governance using fitness functions. Consider the following scenario: what does a company do when a zero-day exploit is discovered in one of the development frameworks or libraries the enterprise uses? If it's like most companies, security experts scour projects to find the offending version of the framework and make sure it's updated, but that process is rarely automated, relying on many manual steps. This isn't an abstract question; this exact scenario affected a major financial institution described in The Equifax Data Breach. Like the architecture governance described previously, manual processes are error prone and allow details to escape.

THE EQUIFAX DATA BREACH

On September 7, 2017, Equifax, a major credit scoring agency in the US, announced that a data breach had occurred. Ultimately, the problem was traced to a hacking exploit of the popular Struts web framework in the Java ecosystem (Apache Struts vCVE-2017-5638). The foundation issued a statement announcing the vulnerability and released a patch on March 7, 2017. The Department of Homeland Security contacted Equifax and similar companies the next day, warning them of this problem, and they ran scans on March 15, 2017, which didn't reveal all of the affected systems. Thus, the critical patch wasn't applied to many older systems until July 29, 2017, when Equifax's security experts identified the hacking behavior that lead to the data breach.

Imagine an alternative world in which every project runs a deployment pipeline, and the security team has a "slot" in each team's deployment pipeline where they can deploy fitness functions. Most of the time, these will be mundane checks for safeguards like preventing developers from storing passwords in databases and similar regular governance chores. However, when a zero-day exploit appears, having the same mechanism in place everywhere allows the security team to insert a test in every project that checks for a certain framework and version number; if it finds the dangerous version, it fails the build and notifies the security team. Teams configure deployment pipelines to awaken for any change to the ecosystem: code, database schema, deployment configuration, and fitness functions. This allows enterprises to universally automate important governance tasks.

Fitness functions provide many benefits for architects, not the least of which is the chance to do some coding again! One of the universal complaints among architects is that they don't get to code much anymore—but fitness functions are often code! By building an executable specification of the architecture, which anyone can validate anytime by running the project's build, architects must understand the system and its ongoing evolution well, which overlaps with the core goal of keeping up with the code of the project as it grows.

However powerful fitness functions are, architects should avoid overusing them. Architects should not form a cabal and retreat to an ivory tower to build an impossibly complex, interlocking set of fitness functions that merely frustrate developers and teams. Instead, it's a way for architects to build an executable checklist of *important* but not *urgent* principles on software projects. Many projects drown in urgency, allowing some important principles to slip by the side. This is the frequent cause of technical debt: "We know this is bad, but we'll come back to fix it later"—and later never comes. By codifying rules about code quality, structure, and other safeguards against decay into

fitness functions that run continually, architects build a quality checklist that developers can't skip.

A few years ago, the excellent book *The Checklist Manifesto* by Atul Gawande (Picador) highlighted the use of checklists by professionals like surgeons, airline pilots, and those other fields who commonly use (sometimes by force of law) checklists as part of their job. It isn't because they don't know their job or are particularly forgetful; when professionals perform the same task over and over, it becomes easy to fool themselves when it's accidentally skipped, and checklists prevent that. Fitness functions represent a checklist of important principles defined by architects and run as part of the build to make sure developers don't accidentally (or purposefully, because of external forces like schedule pressure) skip them.

We utilize fitness functions throughout the book when an opportunity arises to illustrate governing an architectural solution as well as the initial design.

Architecture Versus Design: Keeping Definitions Simple

A constant area of struggle for architects is keeping *architecture* and *design* as separate but related activities. While we don't want to wade into the never-ending argument about this distinction, we strive in this book to stay firmly on the *architecture* side of that spectrum for several reasons.

First, architects must understand underlying architecture principles to make effective decisions. For example, the decision between *synchronous* versus *asynchronous* communication has a number of trade-offs before architects layer in implementation details. In the book *Fundamentals of Software Architecture*, the authors coined the second law of software architecture: *why* is more important than *how*. While ultimately architects must understand how to implement solutions, they must first understand why one choice has better trade-offs than another.

Second, by focusing on architecture concepts, we can avoid the numerous implementations of those concepts. Architects can implement asynchronous communication in a variety of ways; we focus on why an architect would choose asynchronous communication and leave the implementation details to another place.

Third, if we start down the path of implementing all the varieties of options we show, this would be the longest book ever written. Focus on architecture principles allows us

to keep things as generic as they can be.

To keep subjects as grounded in architecture as possible, we use the simplest definitions possible for key concepts. For example, *coupling* in architecture can fill entire books (and it has). To that end, we use the following simple, verging on simplistic, definitions:

Service

In colloquial terms, a *service* is a cohesive collection of functionality deployed as an independent executable. Most of the concepts we discuss with regard to services apply broadly to distributed architectures, and specifically microservices architectures.

In the terms we define in Chapter 2, a *service* is part of an architecture quantum, which includes further definitions of both static and dynamic coupling between services and other quanta.

Coupling

Two artifacts (including services) are coupled if a change in one might require a change in the other to maintain proper functionality.

Component

An architectural building block of the application that does some sort of business or infrastructure function, usually manifested through a package structure (Java), namespace (C#), or a physical grouping of source code files within some sort of directory structure. For example, the component Order History might be implemented through a set of class files located in the namespace `app.business.order.history`.

Synchronous communication

Two artifacts communicate synchronously if the caller must wait for the response before proceeding.

Asynchronous communication

Two artifacts communicate asynchronously if the caller does not wait for the response before proceeding. Optionally, the caller can be notified by the receiver through a separate channel when the request has completed.

Orchestrated coordination

A workflow is orchestrated if it includes a service whose primary responsibility is to coordinate the workflow.

Choreographed coordination

A workflow is choreographed when it lacks an orchestrator; rather, the services in the workflow share the coordination responsibilities of the workflow.

Atomicity

A workflow is *atomic* if all parts of the workflow maintain a consistent state at all times; the opposite is represented by the spectrum of *eventual consistency*, covered in Chapter 6.

Contract

We use the term *contract* broadly to define the interface between two software parts, which may encompass method or function calls, integration architecture remote calls, dependencies, and so on. Anywhere two pieces of software join, a contract is involved.

Software architecture is by its nature abstract: we cannot know what unique combination of platforms, technologies, commercial software, and the other dizzying array of possibilities our readers might have, except that no two are exactly alike. We cover many abstract ideas, but must ground them with some implementation details to make them concrete. To that end, we need a problem to illustrate architecture concepts against—which leads us to the Sysops Squad.

Introducing the Sysops Squad Saga

saga

A long story of heroic achievement.

—Oxford English Dictionary

We discuss a number of sagas in this book, both literal and figurative. Architects have co-opted the term *saga* to describe transactional behavior in distributed architectures (which we cover in detail in Chapter 12). However, discussions about architecture tend to become abstract, especially when considering abstract problems such as the hard parts of architecture. To help solve this problem and provide some real-world context for the solutions we discuss, we kick off a literal saga about the *Sysops Squad*.

We use the Sysops Squad saga within each chapter to illustrate the techniques and trade-offs described in this book. While many books on software architecture cover new development efforts, many real-world problems exist within existing systems. Therefore, our story starts with the existing Sysops Squad architecture highlighted here.

Penultimate Electronics is a large electronics giant that has numerous retail stores throughout the country. When customers buy computers, TVs, stereos, and other electronic equipment, they can choose to purchase a support plan. When problems occur, customer-facing technology experts (the Sysops Squad) come to the customer's residence (or work office) to fix problems with the electronic device.

The four main users of the Sysops Squad ticketing application are as follows:

Administrator

The administrator maintains the internal users of the system, including the list of experts and their corresponding skill set, location, and availability. The administrator also manages all of the billing processing for customers using the system, and maintains static reference data (such as supported products, name-value pairs in the system, and so on).

Customer

The customer registers for the Sysops Squad service and maintains their customer profile, support contracts, and billing information. Customers enter problem tickets into the system, and also fill out surveys after the work has been completed.

Sysops Squad expert

Experts are assigned problem tickets and fix problems based on the ticket. They also interact with the knowledge base to search for solutions to customer problems and enter notes about repairs.

Manager

The manager keeps track of problem ticket operations and receives operational and analytical reports about the overall Sysops Squad problem ticket system.

Nonticketing Workflow

The nonticketing workflows include those actions that administrators, managers, and customers perform that do not relate to a problem ticket. These workflows are outlined as follows:

1. Sysops Squad experts are added and maintained in the system through an administrator, who enters in their locale, availability, and skills.
2. Customers register with the Sysops Squad system and have multiple support plans based on the products they purchased.
3. Customers are automatically billed monthly based on credit card information contained in their profile. Customers can view billing history and statements through the system.
4. Managers request and receive various operational and analytical reports, including financial reports, expert performance reports, and ticketing reports.

Ticketing Workflow

The ticketing workflow starts when a customer enters a problem ticket into the system, and ends when the customer completes the survey after the repair is done. This workflow is outlined as follows:

1. Customers who have purchased the support plan enter a problem ticket by using the Sysops Squad website.
2. Once a problem ticket is entered in the system, the system then determines which Sysops Squad expert would be the best fit for the job based on skills, current location, service area, and availability.
3. Once assigned, the problem ticket is uploaded to a dedicated custom mobile app on the Sysops Squad expert's mobile device. The expert is also notified via a text message that they have a new problem ticket.
4. The customer is notified through an SMS text message or email (based on their profile preference) that the expert is on their way.
5. The expert uses the custom mobile application on their phone to retrieve the ticket information and location. The Sysops Squad expert can also access a knowledge base through the mobile app to find out what has been done in the past to fix the problem.
6. Once the expert fixes the problem, they mark the ticket as "complete." The sysops squad expert can then add information about the problem and repair the knowledge base.
7. After the system receives notification that the ticket is complete, it sends an email to the customer with a link to a survey, which the customer then fills out.
8. The system receives the completed survey from the customer and records the survey information.

A Bad Scenario

Things have not been good with the Sysops Squad problem ticket application lately. The current trouble ticket system is a large monolithic application that was developed many years ago. Customers are complaining that consultants are never showing up because of lost tickets, and often the wrong consultant shows up to fix something they know nothing about. Customers have also been complaining that the system is not always available to enter new problem tickets.

Change is also difficult and risky in this large monolith. Whenever a change is made, it usually takes too long and something else usually breaks. Because of reliability issues, the Sysops Squad system frequently "freezes up," or crashes, resulting in all application functionality not being available anywhere from five minutes to two hours while the problem is identified and the application restarted.

If something isn't done soon, Penultimate Electronics will be forced to abandon the very lucrative support contract business line and lay off all the Sysops Squad administrators, experts, managers, and IT development staff—including the architects.

Sysops Squad Architectural Components

The monolithic system for the Sysops Squad application handles ticket management, operational reporting, customer registration, and billing, as well as general administrative functions such as user maintenance, login, and expert skills and profile maintenance. Figure 1-3 and the corresponding Table 1-1 illustrate and describe the components of the existing monolithic application (the `ss.` part of the namespace specifies the Sysops Squad application context).

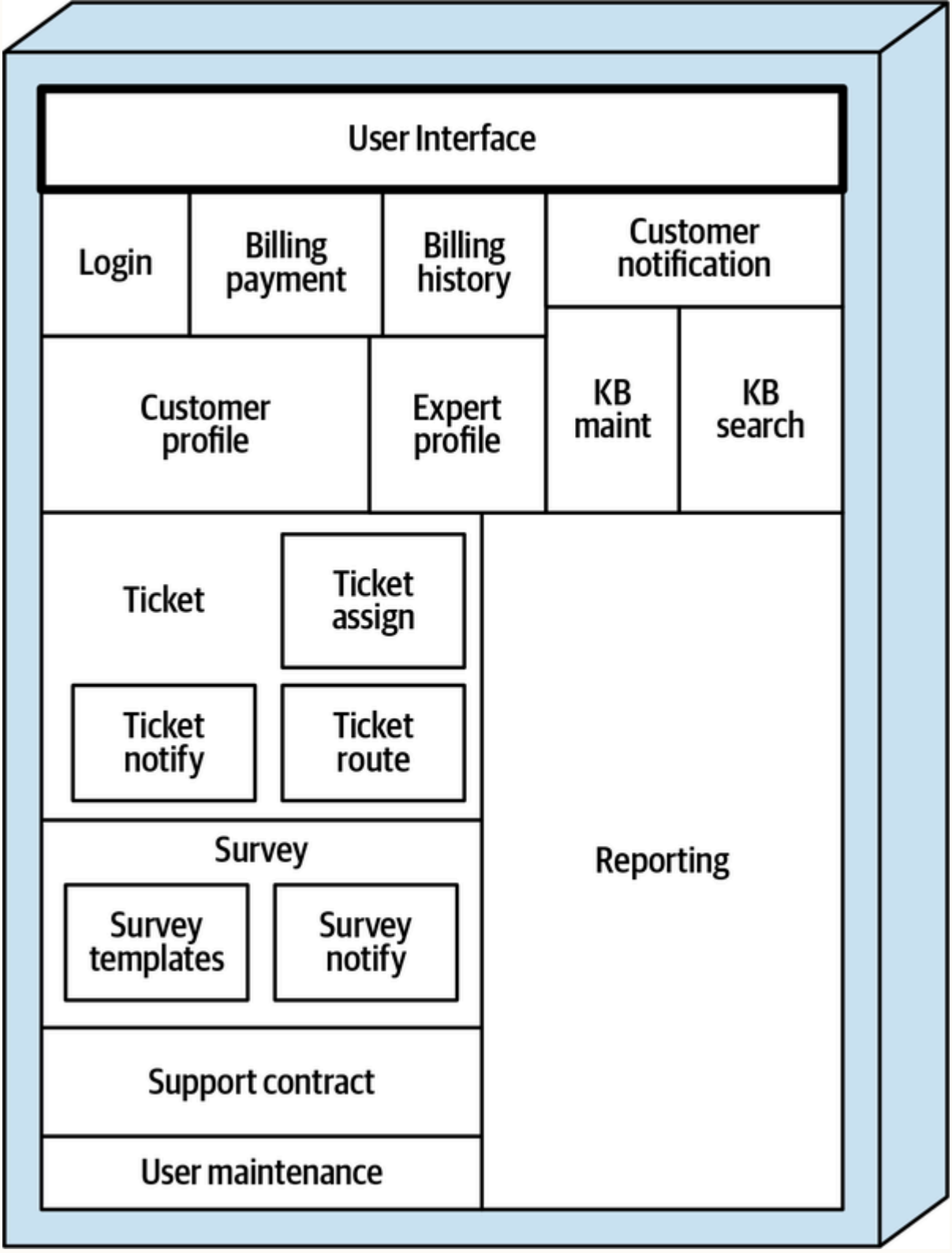


Figure 1-3. Components within the existing Sysops Squad application

Table 1-1. Existing Sysops Squad components

Component	Namespace	Responsibility
Login	ss.login	Internal user and customer login and security logic
Billing payment	ss.billing.payment	Customer monthly billing and customer credit card info
Billing history	ss.billing.history	Payment history and prior billing statements
Customer notification	ss.customer.notification	Notify customer of billing, general info
Customer profile	ss.customer.profile	Maintain customer profile, customer registration
Expert profile	ss.expert.profile	Maintain expert profile (name, location, skills, etc.)
KB maint	ss.kb.maintenance	Maintain and view items in the knowledge base
KB search	ss.kb.search	Query engine for searching the knowledge base
Reporting	ss.reporting	All reporting (experts, tickets, financial)
Ticket	ss.ticket	Ticket creation, maintenance, completion, common code
Ticket assign	ss.ticket.assign	Find an expert and assign the ticket
Ticket notify	ss.ticket.notify	Notify customer that the expert is on their way

Component	Namespace	Responsibility
Ticket route	`ss.ticket.route`	Send the ticket to the expert's mobile device app
Support contract	ss.supportcontract	Support contracts for customers, products in the plan
Survey	`ss.survey`	Maintain surveys, capture and record survey results
Survey notify	ss.survey.notify	Send survey email to customer
Survey templates	`ss.survey.templates`	Maintain various surveys based on type of service
User maintenance	ss.users	Maintain internal users and roles

These components will be used in subsequent chapters to illustrate various techniques and trade-offs when dealing with breaking applications into distributed architectures.

Sysops Squad Data Model

The Sysops Squad application with its various components listed in Table 1-1 uses a single schema in the database to host all its tables and related database code. The database is used to persist customers, users, contracts, billing, payments, knowledge base, and customer surveys; the tables are listed in Table 1-2, and the ER model is illustrated in Figure 1-4.

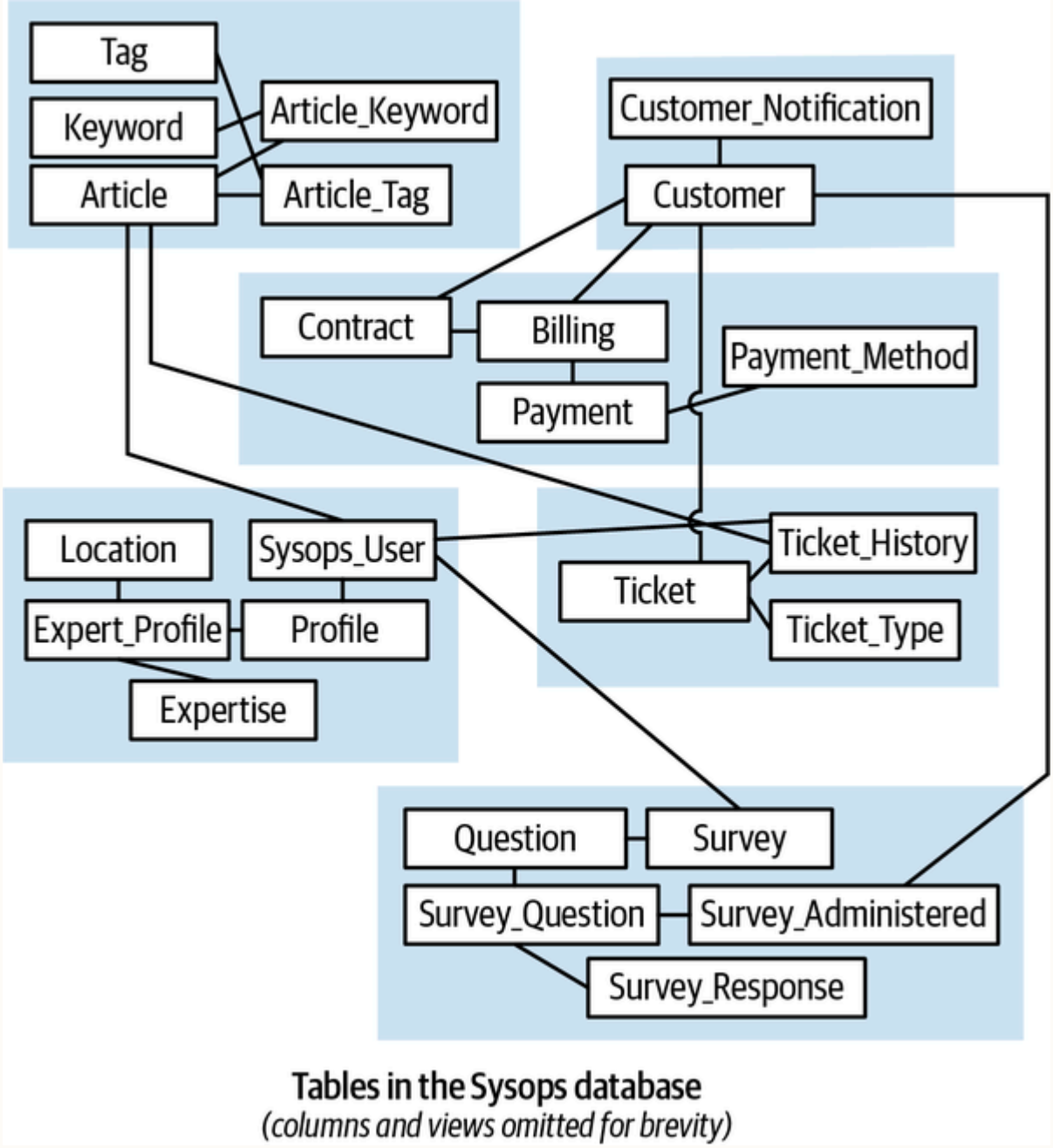


Figure 1-4. Data model within the existing Sysops Squad application

Table 1-2. Existing Sysops Squad database tables

Table	Responsibility
Customer	Entities needing Sysops support
Customer_Notification	Notification preferences for customers
Survey	A survey for after-support customer satisfaction
Question	Questions in a survey
Survey_Question	A question is assigned to the survey
Survey_Administered	Survey question is assigned to customer
Survey_Response	A customer's response to the survey
Billing	Billing information for support contract
Contract	A contract between an entity and Sysops for support
Payment_Method	Payment methods supported for making payment
Payment	Payments processed for billings
SysOps_User	The various users in Sysops
Profile	Profile information for Sysops users
Expert_Profile	Profiles of experts
Expertise	Various expertise within Sysops
Location	Locations served by the expert
Article	Articles for the knowledge base
Tag	Tags on articles

Table	Responsibility
Keyword	Keyword for an article
Article_Tag	Tags associated to articles
Article_Keyword	Join table for keywords and articles
Ticket	Support tickets raised by customers
Ticket_Type	Different types of tickets
Ticket_History	The history of support tickets

The Sysops data model is a standard third normal form data model with only a few stored procedures or triggers. However, a fair number of views exist that are mainly used by the Reporting component. As the architecture team tries to break up the application and move toward distributed architecture, it will have to work with the database team to accomplish the tasks at the database level. This setup of database tables and views will be used throughout the book to discuss various techniques and trade-offs to accomplish the task of breaking apart the database.

Previous chapter
‹ Preface

Next chapter
I. Pulling Things Apart ›